Programming Principles

for (self-taught) front-end developers

I HAVE NO
IDEA WHAT
I'M DOING
PICK ONE

I'm Kilian Valkhof

Kilianvalkhof.com | @kilianvalkhof

I'm Kilian Valkhof

Kilianvalkhof.com | @kilianvalkhof

👴 Spent the past 20 years building websites

I'm Kilian Valkhof

Kilianvalkhof.com | @kilianvalkhof

👴 Spent the past 20 years building websites

🎓 no CS degree

I'm Kilian Valkhof

Kilianvalkhof.com | @kilianvalkhof

👴 Spent the past 20 years building websites

🎓 no CS degree

🌐 Make a browser for devs called Polypane

polypane.app

Polypane, The bro

https://polypane.app

80%

Mobile Phone 100% 320 × 568 px

Polypane

The browser for ambitious developers.

Build better websites in less time with a stand-alone browser that makes your site more responsive, more accessible and faster. Save hours on each project, love every minute of it.

Try Polypane for free →

Free trial. No credit card needed. Cancel anytime.

Small Tablet 100% 500 × 768 px

Polypane

The browser for ambitious developers.

Build better websites in less time with a stand-alone browser that makes your site more responsive, more accessible and faster. Save hours on each project, love every minute of it.

Try Polypane for free →

Free trial. No credit card needed. Cancel anytime.

Laptop 98,73% 1280 × 800 px

Polypane

Features Docs Testimonials Blog Pricing

Try for free →

Info Edit Browse Workspaces

Meta Outline Storage Accessibility Source

Site information

Polypane, The browser for ambitious developers
Build responsive, accessible and fast sites with a stand-alone browser. All the tools you need in one app.

Doctype
<!DOCTYPE html>

Viewport
width=device-width, initial-scale=1, viewport-fit=cover

Mobile Friendly
Yes!

Canonical URL
https://polypane.app/

Charset
utf-8

Language
en

Theme colors
#208dc5 webmanifest
#208dc5

- Robots.txt
- Humans.txt
- Open graph tags
- Twitter tags
- Meta tags
- Webmanifest
- Structured data
- Link tags
- Icons (9)

Rich snippets/social previews

Light Dark

Twitter

Polypane
The browser for ambitious developers

So *many* programming principles

The rule of least power

Choose the least powerful language suitable for a given purpose.

Galls law

A complex system that works is invariably found to have evolved from a simple system that worked.

Hofstadters law

It always takes longer than you expect

Hofstadters law

It always takes longer than you expect, even when you take into account Hofstadter's Law.

Writing better code
as you write code

Live
Laugh
Refactor

Premature optimisation is the root of all evil

Premature
optimisation is the
root of all evil

Premature

optimisation is the

root of all evil

Premature optimisation is **the root of all evil**

Premature optimisation is the root of all evil

YAGNI

You aren't gonna need it

YAGNI

Requirements change, throw away your work

DRY

Don't repeat yourself

You're not gonna need it

but if you do,

you better only write it once

YAGNI & premature optimisation

What do?

Rule of three

Rule of three

Only refactor (or optimize) code once you've written the same code three times.

1st time: Just write it

```
function showSuccess(message) {
  const el = document.createElement('div');
  el.className = 'toast success';
  el.textContent = message;
  document.body.append(el);
  setTimeout(() => el.remove(), 3000);
}
```

2nd time: Copy, make changes

```
function showSuccess(message) {
  const el = document.createElement('div');
  el.className = 'toast success';
  el.textContent = message;
  document.body.append(el);
  setTimeout(() => el.remove(), 3000);
}
```

```
function showError(message) {
  const el = document.createElement('div');
  el.className = 'toast error';
  el.textContent = message;
  document.body.append(el);
  setTimeout(() => el.remove(), 5000);
}
```

3rd time: Refactor

```
function showToast(message, type = 'success', duration = 3000) {
  const el = document.createElement('div');
  el.className = `toast ${type}`;
  el.textContent = message;
  document.body.append(el);
  setTimeout(() => el.remove(), duration);
}
```

Understand your problem before committing to the solution

1. 2. 3.

from

Root of all evil

to

Rule of three

Writing
the right thing

Reading
the right thing

We want it all

Make it work. Make it right. Make it fast.

Kent Beck

Does it work?

```
function formatCartSummary(items) {
            const result = items.reduce((acc, item) => {
              const lineTotal = item.price * item.quantity;
              acc.lines.push("$" + lineTotal);
              acc.cartTotal += lineTotal;
              return acc;
            }, { lines: [], cartTotal: 0 });
            result.cartTotal = "$" + result.cartTotal;
            return result;
          }
```

Is it doing the right thing?

```
function formatCartSummary(items) {
            const result = items.reduce((acc, item) => {
              const lineTotal = item.price * item.quantity;
              const fmt = new Intl.NumberFormat("en-CA",
                { style: "currency", currency: "CAD" });
              acc.lines.push(fmt.format(lineTotal));
              acc.cartTotal += lineTotal;
              return acc;
            }, { lines: [], cartTotal: 0 });
            result.cartTotal = new Intl.NumberFormat("en-CA",
              { style: "currency", currency: "CAD" }).format(resu
            return result;
          }
```

Is it fast enough?

```
function formatCartSummary(items) {
            const fmt = new Intl.NumberFormat("en-CA", { style: "
            const result = { lines: [], cartTotal: 0 };
            for (const item of items) {
              const lineTotal = item.price * item.quantity;
              result.lines.push(fmt.format(lineTotal));
              result.cartTotal += lineTotal;
            }
            result.cartTotal = fmt.format(result.cartTotal);
            return result;
          }
```

Prioritize your efforts

They're the same
picture.

Galls law

A complex system that works is invariably found to have evolved from a simple system that worked.

Galls law (Kilian's version)

Build a crappy first version

Galls law (Kilian's version)

Build a crappy first version

1 make it work, 2 make it right, 3 make it fast

Idempotency

Make function do same thing. Every time

```
"hello".length; // 5
```

```
function deleteUserFromDatabase(userId) {
  try {
    db.delete(userId);
    return userId;
  } catch (e) {
    throw new Error('Could not delete user');
  }
}
```

```
function deleteUserFromDatabase(userId) {
  if (db.has(userId)) {
    db.delete(userId);
  }
  return userId;
}
```

Idempotent

same thing every time

Pure

no side effects

Single responsibility principle

Single responsibility principle

a function (or module, or class) should have "only one reason to change."

one function/class/etc
takes care of
one aspect
of your system

a function/class/etc
should have only
one reason
to exist

No "And"

One level of abstraction

...kind of an abstract concept

```javascript
async function processUsers() {
  const users = await database.fetchAllUsers();

  users.forEach((user) => {
    if (user.isActive) {
      sendEmail(user.email, "Hello active user!");
    }
  });
}
```

```javascript
async function processUsers() {
  const users = await database.fetchAllUsers();

  users.forEach((user) => {
    if (user.isActive) {
      sendEmail(user.email, "Hello active user!");
    }
  });
}
```

```javascript
async function getActiveUsers() {
  const users = await database.fetchAllUsers();
  return users.filter((user) => user.isActive);
}
function sendEmailsToUsers(users) {
  users.forEach((user) => {
    sendEmail(user.email, "Hello active user!");
  });
}
async function processUsers() {
  const activeUsers = await getActiveUsers();
  sendEmailsToUsers(activeUsers);
}
```

All good code is alike; each piece of bad code is bad in its own way.

Leo Tolstoy, renowned software engineer

Quick end-to-end example

1) Work

```
const getTotal = (items) =>
  "$" + items.reduce((sum, i) => sum + i.price * i.quantity, 0);
```

Quick end-to-end example

2) Right

```
const calcTotal = (items) =>
  items.reduce((sum, i) => sum + i.price * i.quantity, 0);

const formatTotal = (total, locale = "en-CA", currency = "CAD") =
  new Intl.NumberFormat(locale, { style: "currency", currency }).

const getCartTotal = (items) => formatTotal(calcTotal(items));
```

Quick end-to-end example

3) Fast

```
const fmt = new Intl.NumberFormat("en-CA", { style: "currency", c
const calcTotal = (items) =>
  items.reduce((sum, i) => sum + i.price * i.quantity, 0);
const formatTotalFast = (total) => fmt.format(total);
const getCartTotalFast = (items) => formatTotalFast(calcTotal(ite
```

Rule of three

Make it work, make it right, make it fast

Build a crappy first version

Idempotency

Single responsibility principle

One level of abstraction

Thank you!

Kilianvalkhof.com | @kilianvalkhof

Further reading

- Refactoring by Martin Fowler.
- Making impossible states impossible by Richard Feldman.
- Make It Work Make It Right Make It Fast on the C2 wiki.
- Software Design Principles: Single Level of Abstraction.
- hacker-laws.com.